

LoRA: 大语言模型的低秩自适应

(LoRA: Low-Rank Adaptation of Large Language Models)

Edward Hu* Yelong Shen* Phillip Wallis Zeyuan Allen-Zhu

Yuanzhi Li Shean Wang Lu Wang Weizhu Chen

Microsoft Corporation (微软公司)

{edwardhu, yeshe, phwallis, zeyuana,

yuanzhil, swang, luw, wzchen}@microsoft.com

yuanzhil@andrew.cmu.edu

(Version 2, 第 2 版; arXiv:2106.09685v2 [cs.CL], 2021 年 10 月 16 日)

摘要

自然语言处理的一个重要范式是：先在通用领域数据上进行大规模预训练 (pre-training)，再对特定任务或领域进行自适应 (adaptation)。随着我们预训练的模型越来越大，重新训练全部模型参数的完全微调 (full fine-tuning) 变得愈发不可行。以 GPT-3 175B 为例——为每个任务部署各自独立的、各含 175B 参数的微调模型实例，其代价高得令人望而却步。我们提出低秩自适应 (Low-Rank Adaptation, 简称 LoRA)，它冻结预训练模型权重，并在 Transformer 架构的每一层中注入可训练的秩分解矩阵 (rank decomposition matrices)，从而大幅减少下游任务的可训练参数数量。与使用 Adam 微调的 GPT-3 175B 相比，LoRA 可将可训练参数数量减少 10,000 倍，并将 GPU 显存需求减少 3 倍。在 RoBERTa、DeBERTa、GPT-2 和 GPT-3 上，尽管 LoRA 的可训练参数更少、训练吞吐量更高，且与 adapter 不同地不引入额外的推理延迟 (inference latency)，其模型质量却与微调持平甚至更优。我们还对语言模型自适应中的秩亏 (rank-deficiency) 现象进行了实证研究，从而揭示了 LoRA 的有效性。我们发布了一个便于将 LoRA 与 PyTorch 模型集成的软件包，并在 <https://github.com/microsoft/LoRA> 提供了我们针对 RoBERTa、DeBERTa 和 GPT-2 的实现与模型检查点 (model checkpoints)。

1 引言 (Introduction)

自然语言处理中的许多应用都依赖于将一个大规模、预训练的语言模型自适应到多个下游应用。这种自适应通常通过微调来完成，即更新预训练模型的全部参数。微调的主要缺点在于，新模型所含

*共同贡献，按字母顺序排列。

的参数量与原始模型一样多。随着每隔几个月就会训练出更大的模型，这一问题从 GPT-2 (Radford et al., b) 或 RoBERTa large (Liu et al., 2019) 时代仅仅是一种“不便”，转变为对拥有 1750 亿可训练参数的 GPT-3 (Brown et al., 2020) 而言的关键部署难题。¹

许多人试图通过仅自适应部分参数，或为新任务学习外部模块来缓解这一问题。这样一来，对于每个任务，我们只需在预训练模型之外存储并加载少量任务专属参数，从而在部署时大幅提升运行效率。然而，现有技术往往会通过增加模型深度而引入推理延迟 (Houlsby et al., 2019; Rebuffi et al., 2017)，或者减少模型可用的序列长度 (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021) (见第 3 节)。更重要的是，这些方法往往无法达到微调基线的水平，从而在效率与模型质量之间形成了一种取舍。

我们从 Li et al. (2018a) 与 Aghajanyan et al. (2020) 中获得了启发，他们表明：学习得到的过参数化 (over-parametrized) 模型实际上驻留于一个低内在维度 (low intrinsic dimension) 之上。我们假设：在模型自适应过程中，权重的变化同样具有较低的“内在秩 (intrinsic rank)”，这便引出了我们提出的低秩自适应 (LoRA) 方法。LoRA 使我们能够通过优化稠密层 (dense layer) 在自适应期间变化量的秩分解矩阵，来间接地训练神经网络中的某些稠密层，同时保持预训练权重冻结，如图 1 所示。以 GPT-3 175B 为例，我们表明：即使完整秩 (即 d) 高达 12,288，一个非常低的秩 (即图 1 中的 r 可以为一或二) 也已足够，这使得 LoRA 在存储和计算上都很高效。

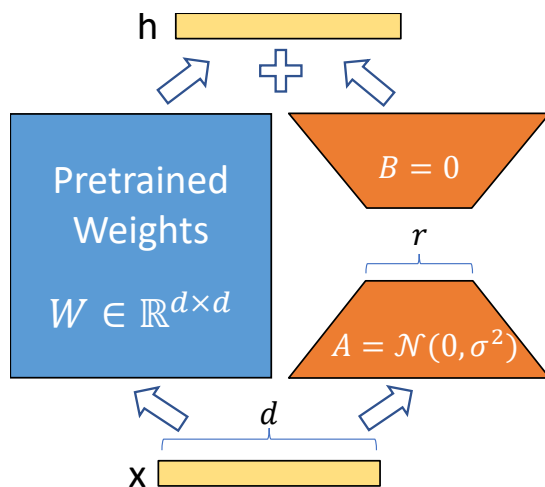


图 1: 我们的重参数化 (reparametrization)。我们只训练 A 和 B 。其中预训练权重 $W \in \mathbb{R}^{d \times d}$ 被冻结, $A = \mathcal{N}(0, \sigma^2)$, $B = 0$, 秩为 r , 输入为 x , 输出为 $h = f(x)$ 。

LoRA 具有若干关键优势。

- 一个预训练模型可以被共享，并用于为不同任务构建许多小型 LoRA 模块。我们可以冻结这个共享模型，并通过替换图 1 中的矩阵 A 和 B 来高效地切换任务，从而显著降低存储需求和任务切换的开销。

¹尽管 GPT-3 175B 通过少样本学习 (few-shot learning) 能取得不俗的性能，但如附录 A 所示，微调能显著提升其性能。

- 当使用自适应优化器 (adaptive optimizers) 时, LoRA 使训练更高效, 并将硬件准入门槛最多降低 3 倍, 因为我们无需为大多数参数计算梯度或维护优化器状态。相反, 我们只优化注入的、规模小得多的低秩矩阵。
- 我们简单的线性设计使我们能够在部署时将可训练矩阵与冻结权重合并, 从结构上保证与完全微调的模型相比不引入任何推理延迟。
- LoRA 与许多先前的方法正交, 并且可以与其中许多方法相结合, 例如前缀微调 (prefix-tuning)。我们在附录 E 中给出了一个示例。

术语与约定 (Terminologies and Conventions) 我们将频繁提及 Transformer 架构, 并使用其维度的常规术语。我们将 Transformer 层的输入和输出维度大小称为 d_{model} 。我们用 W_q 、 W_k 、 W_v 和 W_o 来分别指代自注意力 (self-attention) 模块中的查询/键/值/输出投影矩阵 (query/key/value/output projection matrices)。 W 或 W_0 指代预训练权重矩阵, ΔW 指代其在自适应期间累积的梯度更新。我们用 r 表示一个 LoRA 模块的秩。我们遵循 (Vaswani et al., 2017; Brown et al., 2020) 所确立的约定, 使用 Adam (Loshchilov & Hutter, 2019; Kingma & Ba, 2017) 进行模型优化, 并使用 Transformer MLP 前馈维度 $d_{ffn} = 4 \times d_{model}$ 。

2 问题陈述 (Problem Statement)

虽然我们的提案与训练目标无关 (agnostic to training objective), 但我们将语言建模 (language modeling) 作为驱动性的应用场景来重点讨论。下面简要描述语言建模问题, 特别是在给定任务专属提示 (task-specific prompt) 的情况下对条件概率的最大化。

假设给定一个由 Φ 参数化的预训练自回归语言模型 $P_\Phi(y|x)$ 。例如, $P_\Phi(y|x)$ 可以是一个基于 Transformer 架构 (Vaswani et al., 2017) 的通用多任务学习器, 如 GPT (Radford et al., b; Brown et al., 2020)。考虑将这个预训练模型自适应到下游条件文本生成任务, 如摘要 (summarization)、机器阅读理解 (machine reading comprehension, MRC) 以及自然语言到 SQL (natural language to SQL, NL2SQL)。每个下游任务由一个上下文-目标对 (context-target pairs) 的训练数据集表示: $Z = \{(x_i, y_i)\}_{i=1, \dots, N}$, 其中 x_i 和 y_i 都是 token 序列。例如, 在 NL2SQL 中, x_i 是一条自然语言查询, y_i 是其对应的 SQL 命令; 对于摘要任务, x_i 是一篇文章的内容, y_i 是其摘要。

在完全微调期间, 模型被初始化为预训练权重 Φ_0 , 并通过反复沿梯度方向以最大化条件语言建模目标, 更新为 $\Phi_0 + \Delta\Phi$:

$$\max_{\Phi} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log(P_\Phi(y_t|x, y_{<t})) \quad (1)$$

完全微调的主要缺点之一是: 对于每个下游任务, 我们都要学习一组不同的参数 $\Delta\Phi$, 其维度 $|\Delta\Phi|$ 等于 $|\Phi_0|$ 。因此, 如果预训练模型很大 (例如 GPT-3, $|\Phi_0| \approx 1750$ 亿), 那么存储和部署许多独立的微调模型实例将极具挑战性, 甚至根本不可行。

本文采用一种参数效率更高 (parameter-efficient) 的方法, 其中任务专属的参数增量 $\Delta\Phi = \Delta\Phi(\Theta)$ 被进一步用一组规模小得多的参数 Θ ($|\Theta| \ll |\Phi_0|$) 来编码。因此, 寻找 $\Delta\Phi$ 的任务就变成了对 Θ 进行优化:

$$\max_{\Theta} \sum_{(x,y) \in Z} \sum_{t=1}^{|y|} \log(p_{\Phi_0 + \Delta\Phi(\Theta)}(y_t | x, y_{<t})) \quad (2)$$

在后续各节中, 我们提出使用一种低秩表示来编码 $\Delta\Phi$, 它在计算和内存上都很高效。当预训练模型为 GPT-3 175B 时, 可训练参数数量 $|\Theta|$ 可以小到 $|\Phi_0|$ 的 0.01%。

3 现有方案足够好了吗？(Aren't Existing Solutions Good Enough?)

我们着手解决的问题绝非新问题。自从迁移学习 (transfer learning) 诞生以来, 已有数十项工作试图使模型自适应在参数和计算上更高效。第 6 节对一些著名工作进行了综述。以语言建模为例, 在高效自适应方面有两种主流策略: 添加 adapter 层 (Houlsby et al., 2019; Rebuffi et al., 2017; Pfeiffer et al., 2021; Rücklé et al., 2020), 或优化某种形式的输入层激活 (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021)。然而, 这两种策略都有其局限性, 尤其是在大规模且延迟敏感的生产场景中。

Adapter 层引入推理延迟 adapter 有许多变体。我们关注 Houlsby et al. (2019) 的原始设计——每个 Transformer 块 (block) 有两个 adapter 层; 以及 Lin et al. (2020) 最近的一种设计——每个块只有一个 adapter 层, 但附加了一个 LayerNorm (Ba et al., 2016)。虽然可以通过剪枝层或利用多任务设定 (Rücklé et al., 2020; Pfeiffer et al., 2021) 来降低总体延迟, 但没有直接的方法能够绕过 adapter 层中的额外计算。这看起来似乎不成问题, 因为 adapter 层被设计为通过较小的瓶颈维度 (bottleneck dimension) 来仅含少量参数 (有时不到原模型的 1%), 从而限制了它们所能增加的浮点运算量 (FLOPs)。然而, 大型神经网络依赖硬件并行性 (hardware parallelism) 来保持低延迟, 而 adapter 层必须被顺序处理。这在线推理 (online inference) 场景中会带来显著差异, 因为该场景中的批大小 (batch size) 通常小到只有 1。在没有模型并行性 (model parallelism) 的通用场景中, 例如在单个 GPU 上对 GPT-2 (Radford et al., b) medium 进行推理, 我们观察到使用 adapter 时延迟有明显增加, 即便瓶颈维度非常小 (见表 1)。

当我们需要像 Shoeybi et al. (2020)、Lepikhin et al. (2020) 那样对模型进行分片 (shard) 时, 这个问题会变得更糟, 因为额外的深度需要更多同步的 GPU 操作, 例如 AllReduce 和 Broadcast, 除非我们将 adapter 参数冗余地存储许多份。

直接优化提示是困难的 另一个方向, 以前缀微调 (prefix tuning) (Li & Liang, 2021) 为代表, 面临着不同的挑战。我们观察到前缀微调难以优化, 且其性能随可训练参数变化呈非单调 (non-monotonically) 的趋势, 这印证了原论文中类似的观察结果。更根本地说, 为自适应保留一部分序列长度, 必然会减少可用于处理下游任务的序列长度, 我们怀疑这使得调整提示 (tuning the prompt) 相比其他方法性

能更差。我们将任务性能的研究推迟到第 5 节。

表 1: GPT-2 medium 中单次前向传递 (forward pass) 的推理延迟, 以毫秒为单位测量, 对 100 次试验取平均。我们使用一块 NVIDIA Quadro RTX8000。“ $|\Theta|$ ” 表示 adapter 层中的可训练参数数量。Adapter^L 和 Adapter^H 是 adapter 微调的两种变体, 我们在 5.1 节中描述。adapter 层引入的推理延迟在在线、短序列长度的场景中可能十分显著。完整研究见附录 B。

批大小 (Batch Size)	32	16	1
序列长度 (Sequence Length)	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4 $\pm$ 0.8	338.0 $\pm$ 0.6	19.8 $\pm$ 2.7
Adapter ^L	1482.0 $\pm$ 1.0 (+2.2%)	354.8 $\pm$ 0.5 (+5.0%)	23.9 $\pm$ 2.1 (+20.7%)
Adapter ^H	1492.2 $\pm$ 1.0 (+3.0%)	366.3 $\pm$ 0.5 (+8.4%)	25.8 $\pm$ 2.2 (+30.3%)

4 我们的方法 (Our Method)

我们描述 LoRA 简单的设计及其实际收益。这里给出的原理适用于深度学习模型中的任意稠密层, 不过作为驱动性的应用场景, 我们在实验中只关注 Transformer 语言模型中的某些权重。

4.1 低秩参数化的更新矩阵 (Low-Rank-Parametrized Update Matrices)

神经网络包含许多执行矩阵乘法的稠密层。这些层中的权重矩阵通常是满秩 (full-rank) 的。当自适应到某个特定任务时, Aghajanyan et al. (2020) 表明, 预训练语言模型具有较低的“内在维度”, 即便被随机投影到一个更小的子空间, 仍能高效地学习。受此启发, 我们假设权重在自适应期间的更新也具有较低的“内在秩”。对于一个预训练权重矩阵 $W_0 \in \mathbb{R}^{d \times k}$, 我们通过用一个低秩分解来表示其更新, 从而对该更新加以约束: $W_0 + \Delta W = W_0 + BA$, 其中 $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, 且秩 $r \ll \min(d, k)$ 。在训练期间, W_0 被冻结, 不接受梯度更新, 而 A 和 B 包含可训练参数。注意 W_0 和 $\Delta W = BA$ 都与同一个输入相乘, 且它们各自的输出向量按坐标逐位相加。对于 $h = W_0 x$, 我们修改后的前向传递得到:

$$h = W_0 x + \Delta W x = W_0 x + BAx \quad (3)$$

我们在图 1 中说明了这种重参数化。我们对 A 使用随机高斯初始化 (random Gaussian initialization), 对 B 使用零初始化, 因此在训练开始时 $\Delta W = BA$ 为零。然后我们将 $\Delta W x$ 按 $\frac{\alpha}{r}$ 进行缩放, 其中 α 是关于 r 的一个常数。当使用 Adam 优化时, 如果我们适当地缩放初始化, 那么调整 α 与调整学习率 (learning rate) 大致相同。因此, 我们简单地将 α 设为我们尝试的第一个 r , 并且不再调整它。这种缩放有助于减少在我们改变 r 时重新调整超参数的需要 (Yang & Hu, 2021)。

完全微调的一种推广 (A Generalization of Full Fine-tuning) 一种更一般形式的微调允许训练预训练参数的一个子集。LoRA 更进一步，它不要求权重矩阵的累积梯度更新在自适应期间具有满秩。这意味着，当将 LoRA 应用于所有权重矩阵并训练所有偏置 (bias)²时，通过将 LoRA 的秩 r 设为预训练权重矩阵的秩，我们大致可以恢复完全微调的表达能力。换言之，随着我们增加可训练参数的数量³，训练 LoRA 大致收敛于训练原始模型，而基于 adapter 的方法收敛于一个 MLP，基于前缀的方法则收敛于一个无法接受长输入序列的模型。

无额外推理延迟 (No Additional Inference Latency) 当部署到生产环境时，我们可以显式地计算并存储 $W = W_0 + BA$ ，然后照常进行推理。注意 W_0 和 BA 都在 $\mathbb{R}^{d \times k}$ 中。当我们需要切换到另一个下游任务时，可以通过减去 BA 来恢复 W_0 ，然后加上一个不同的 $B'A'$ ，这是一个快速的操作，内存开销极小。关键在于，这从结构上保证了我们在推理时相比微调模型不会引入任何额外延迟。

4.2 将 LoRA 应用于 Transformer (Applying LoRA to Transformer)

原则上，我们可以将 LoRA 应用于神经网络中权重矩阵的任意子集，以减少可训练参数的数量。在 Transformer 架构中，自注意力模块中有四个权重矩阵 (W_q 、 W_k 、 W_v 、 W_o)，MLP 模块中有两个。我们将 W_q (或 W_k 、 W_v) 视为一个维度为 $d_{model} \times d_{model}$ 的单一矩阵，尽管其输出维度通常被切分为多个注意力头 (attention heads)。出于简单性和参数效率的考虑，我们将研究限制为仅针对下游任务自适应注意力权重，并冻结 MLP 模块 (因此它们不在下游任务中训练)。我们在 7.1 节进一步研究在 Transformer 中自适应不同类型注意力权重矩阵的效果。我们将自适应 MLP 层、LayerNorm 层和偏置的实证研究留待未来工作。

实际收益与局限 (Practical Benefits and Limitations) 最显著的收益来自内存和存储使用的减少。对于一个用 Adam 训练的大型 Transformer，如果 $r \ll d_{model}$ ，由于我们无需为冻结参数存储优化器状态，我们最多可将该显存 (VRAM) 使用量减少 2/3。在 GPT-3 175B 上，我们将训练期间的显存消耗从 1.2TB 减少到 350GB。当 $r = 4$ 且仅自适应查询和值投影矩阵时，检查点大小减少了大约 10,000 倍 (从 350GB 减少到 35MB)⁴。这使我们能够用显著更少的 GPU 进行训练，并避免 I/O 瓶颈。另一个收益是，在部署时我们可以以低得多的成本在任务之间切换，只需替换 LoRA 权重而非全部参数。这使得我们能够创建许多定制化模型，并在将预训练权重存储于显存的机器上即时换入换出。我们还观察到，在 GPT-3 175B 上的训练相比完全微调有 25% 的加速⁵，因为我们无需为绝大多数参数计算梯度。

LoRA 也有其局限性。例如，如果选择将 A 和 B 吸收进 W 以消除额外的推理延迟，那么在单次

²与权重相比，它们所代表的参数数量可以忽略不计。

³在自适应到困难任务时这是不可避免的。

⁴在部署期间我们仍然需要 350GB 的模型；然而，存储 100 个自适应后的模型仅需 $350\text{GB} + 35\text{MB} \times 100 \approx 354\text{GB}$ ，而非 $100 \times 350\text{GB} \approx 35\text{TB}$ 。

⁵对于 GPT-3 175B，完全微调的训练吞吐量为每块 V100 GPU 32.5 tokens/s；在模型并行使用相同权重分片数量的情况下，LoRA 的吞吐量为每块 V100 GPU 43.1 tokens/s。

前向传递中将不同任务的输入（各自使用不同的 A 和 B ）进行批处理就不那么直接了。不过，在延迟并非关键的场景中，也可以不合并权重，而是动态地为一个批次中的样本选择要使用的 LoRA 模块。

5 实证实验 (Empirical Experiments)

我们在 RoBERTa (Liu et al., 2019)、DeBERTa (He et al., 2021) 和 GPT-2 (Radford et al., b) 上评估 LoRA 的下游任务性能，然后再扩展到 GPT-3 175B (Brown et al., 2020)。我们的实验涵盖了广泛的任务，从自然语言理解 (natural language understanding, NLU) 到生成 (natural language generation, NLG)。具体而言，我们在 GLUE (Wang et al., 2019) 基准上评估 RoBERTa 和 DeBERTa。我们在 GPT-2 上遵循 Li & Liang (2021) 的设置以进行直接比较，并为 GPT-3 上的大规模实验加入 WikiSQL (Zhong et al., 2017) (自然语言到 SQL 查询) 和 SAMSum (Gliwa et al., 2019) (对话摘要)。关于我们所用数据集的更多细节见附录 C。所有实验我们都使用 NVIDIA Tesla V100。

5.1 基线 (Baselines)

为了与其他基线进行广泛比较，我们复现了先前工作所采用的设置，并尽可能复用它们报告的数字。然而，这意味着某些基线可能只出现在特定实验中。

微调 (Fine-Tuning, FT) 是一种常见的自适应方法。在微调期间，模型被初始化为预训练的权重和偏置，且所有模型参数都进行梯度更新。一个简单的变体是只更新某些层而冻结其他层。我们纳入了先前工作 (Li & Liang, 2021) 在 GPT-2 上报告的一个这样的基线，它仅自适应最后两层 (FT^{Top2})。

仅偏置 (Bias-only) 或 BitFit 是一个仅训练偏置向量而冻结其余一切的基线。在同一时期，这一基线也被 BitFit (Zaken et al., 2021) 所研究。

前缀嵌入微调 (Prefix-embedding tuning, PreEmbed) 在输入 token 之间插入特殊 token。这些特殊 token 具有可训练的词嵌入 (word embeddings)，并且一般不在模型的词表中。这些 token 放置的位置会影响性能。我们关注“前置 (prefixing)”，即将这些 token 加在提示之前；以及“中插 (infixing)”，即将其追加到提示之后；二者在 Li & Liang (2021) 中都有讨论。我们用 l_p (相应地 l_i) 表示前缀 (相应地中插) token 的数量。可训练参数的数量为 $|\Theta| = d_{model} \times (l_p + l_i)$ 。

前缀层微调 (Prefix-layer tuning, PreLayer) 是前缀嵌入微调的一种扩展。我们不再只学习某些特殊 token 的词嵌入 (或等价地，嵌入层之后的激活)，而是学习每个 Transformer 层之后的激活。由先前层计算得到的激活被简单地替换为可训练的激活。由此得到的可训练参数数量为 $|\Theta| = L \times d_{model} \times (l_p + l_i)$ ，其中 L 是 Transformer 层的数量。

Adapter 微调 (Adapter tuning) 如 Houlsby et al. (2019) 所提出的, 在自注意力模块 (及 MLP 模块) 与随后的残差连接 (residual connection) 之间插入 adapter 层。一个 adapter 层中有两个带偏置的全连接层, 其间夹有一个非线性。我们将这一原始设计称为 Adapter^H。最近, Lin et al. (2020) 提出了一种更高效的设计, 其中 adapter 层仅应用于 MLP 模块之后以及一个 LayerNorm 之后。我们将其称为 Adapter^L。这与 Pfeiffer et al. (2021) 提出的另一种设计非常相似, 我们将后者称为 Adapter^P。我们还纳入了另一个名为 AdapterDrop (Rücklé et al., 2020) 的基线, 它为了更高的效率而丢弃某些 adapter 层 (Adapter^D)。我们尽可能从先前工作中引用数字, 以最大化我们所比较的基线数量; 它们位于第一列带星号 (*) 的行中。在所有情况下, 我们都有 $|\Theta| = \hat{L}_{Adpt} \times (2 \times d_{model} \times r + r + d_{model}) + 2 \times \hat{L}_{LN} \times d_{model}$, 其中 $\hat{L}_{Adpt}$ 是 adapter 层的数量, $\hat{L}_{LN}$ 是可训练 LayerNorm 的数量 (例如在 Adapter^L 中)。

LoRA 在现有权重矩阵旁并行地添加可训练的秩分解矩阵对。如 4.2 节所述, 为简单起见, 在大多数实验中我们只将 LoRA 应用于 W_q 和 W_v 。可训练参数的数量由秩 r 和原始权重的形状决定: $|\Theta| = 2 \times \hat{L}_{LoRA} \times d_{model} \times r$, 其中 $\hat{L}_{LoRA}$ 是我们应用 LoRA 的权重矩阵数量。

表 2: 在 GLUE 基准上采用不同自适应方法的 RoBERTa_{base}、RoBERTa_{large} 和 DeBERTa_{XXL}。我们报告 MNLI 的总体 (匹配与不匹配) 准确率、CoLA 的 Matthew 相关系数、STS-B 的 Pearson 相关系数, 以及其他任务的准确率。对所有指标而言越高越好。* 表示先前工作发表的数字。† 表示在与 Houlsby et al. (2019) 类似的设置下配置的运行, 以进行公平比较。

模型与方法	参数量	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
RoB _{base} (FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2	86.4
RoB _{base} (BitFit)*	0.1M	84.7	93.7	92.7	62.0	91.8	84.0	81.5	90.8	85.2
RoB _{base} (Adpt ^D)*	0.3M	87.1 ± .0	94.2 ± .1	88.5 ± 1.1	60.8 ± .4	93.1 ± .1	90.2 ± .0	71.5 ± 2.7	89.7 ± .3	84.4
RoB _{base} (Adpt ^D)*	0.9M	87.3 ± .1	94.7 ± .3	88.4 ± .1	62.6 ± .9	93.0 ± .2	90.6 ± .0	75.9 ± 2.2	90.3 ± .1	85.4
RoB _{base} (LoRA)	0.3M	87.5 ± .3	95.1 ± .2	89.7 ± .7	63.4 ± 1.2	93.3 ± .3	90.8 ± .1	86.6 ± .7	91.5 ± .2	87.2
RoB _{large} (FT)*	355.0M	90.2	96.4	90.9	68.0	94.7	92.2	86.6	92.4	88.9
RoB _{large} (LoRA)	0.8M	90.6 ± .2	96.2 ± .5	90.9 ± 1.2	68.2 ± 1.9	94.9 ± .3	91.6 ± .1	87.4 ± 2.5	92.6 ± .2	89.0
RoB _{large} (Adpt ^P)†	3.0M	90.2 ± .3	96.1 ± .3	90.2 ± .7	68.3 ± 1.0	94.8 ± .2	91.9 ± .1	83.8 ± 2.9	92.1 ± .7	88.4
RoB _{large} (Adpt ^P)†	0.8M	90.5 ± .3	96.6 ± .2	89.7 ± 1.2	67.8 ± 2.5	94.8 ± .3	91.7 ± .2	80.1 ± 2.9	91.9 ± .4	87.9
RoB _{large} (Adpt ^H)†	6.0M	89.9 ± .5	96.2 ± .3	88.7 ± 2.9	66.5 ± 4.4	94.7 ± .2	92.1 ± .1	83.4 ± 1.1	91.0 ± 1.7	87.8
RoB _{large} (Adpt ^H)†	0.8M	90.3 ± .3	96.3 ± .5	87.7 ± 1.7	66.3 ± 2.0	94.7 ± .2	91.5 ± .1	72.9 ± 2.9	91.5 ± .5	86.4
RoB _{large} (LoRA)†	0.8M	90.6 ± .2	96.2 ± .5	90.2 ± 1.0	68.2 ± 1.9	94.8 ± .3	91.6 ± .2	85.2 ± 1.1	92.3 ± .5	88.6
DeB _{XXL} (FT)*	1500.0M	91.8	97.2	92.0	72.0	96.0	92.7	93.9	92.9	91.1
DeB _{XXL} (LoRA)	4.7M	91.9 ± .2	96.9 ± .2	92.6 ± .6	72.4 ± 1.1	96.0 ± .1	92.9 ± .1	94.9 ± .4	93.0 ± .2	91.3

表 3: 在 E2E NLG Challenge 上采用不同自适应方法的 GPT-2 medium (M) 和 large (L)。对所有指标而言越高越好。在可训练参数相当或更少的情况下, LoRA 优于若干基线。对于我们自己运行的实验给出了置信区间。
* 表示先前工作发表的数字。

模型与方法	参数量	BLEU	NIST	MET	ROUGE-L	CIDEr
GPT-2 M (FT)*	354.92M	68.2	8.62	46.2	71.0	2.47
GPT-2 M (Adapter ^L)*	0.37M	66.3	8.41	45.0	69.8	2.40
GPT-2 M (Adapter ^L)*	11.09M	68.9	8.71	46.1	71.3	2.47
GPT-2 M (Adapter ^H)	11.09M	67.3 $\pm$.6	8.50 $\pm$.07	46.0 $\pm$.2	70.7 $\pm$.2	2.44 $\pm$.01
GPT-2 M (FT ^{Top2})*	25.19M	68.1	8.59	46.0	70.8	2.41
GPT-2 M (PreLayer)*	0.35M	69.7	8.81	46.1	71.4	2.49
GPT-2 M (LoRA)	0.35M	70.4 $\pm$.1	8.85 $\pm$.02	46.8 $\pm$.2	71.8 $\pm$.1	2.53 $\pm$.02
GPT-2 L (FT)*	774.03M	68.5	8.78	46.0	69.9	2.45
GPT-2 L (Adapter ^L)	0.88M	69.1 $\pm$.1	8.68 $\pm$.03	46.3 $\pm$.0	71.4 $\pm$.2	2.49 $\pm$.0
GPT-2 L (Adapter ^L)	23.00M	68.9 $\pm$.3	8.70 $\pm$.04	46.1 $\pm$.1	71.3 $\pm$.2	2.45 $\pm$.02
GPT-2 L (PreLayer)*	0.77M	70.3	8.85	46.2	71.7	2.47
GPT-2 L (LoRA)	0.77M	70.4 $\pm$.1	8.89 $\pm$.02	46.8 $\pm$.2	72.0 $\pm$.2	2.47 $\pm$.02

5.2 RoBERTa base/large

RoBERTa (Liu et al., 2019) 优化了最初在 BERT (Devlin et al., 2019a) 中提出的预训练方案, 在不引入过多可训练参数的情况下提升了后者的任务性能。尽管近年来在诸如 GLUE 基准 (Wang et al., 2019) 等 NLP 排行榜上, RoBERTa 已被大得多的模型超越, 但就其规模而言, 它在从业者中仍是一个有竞争力且受欢迎的预训练模型。我们从 HuggingFace Transformers 库 (Wolf et al., 2020) 中取得预训练的 RoBERTa base (125M) 和 RoBERTa large (355M), 并在 GLUE 基准的任务上评估不同高效自适应方法的性能。我们还根据 Houlsby et al. (2019) 和 Pfeiffer et al. (2021) 的设置复现了它们的工作。为确保公平比较, 在与 adapter 比较时, 我们对评估 LoRA 的方式做了两处关键改动。第一, 我们对所有任务使用相同的批大小, 并使用 128 的序列长度, 以匹配 adapter 基线。第二, 对于 MRPC、RTE 和 STS-B, 我们将模型初始化为预训练模型, 而非像微调基线那样初始化为一个已经自适应到 MNLI 的模型。遵循 Houlsby et al. (2019) 这一更受限设置的运行用 $\dagger$ 标记。结果呈现在表 2 (上三部分) 中。所用超参数的细节见 D.1 节。

5.3 DeBERTa XXL

DeBERTa (He et al., 2021) 是 BERT 的一个较新变体, 它在大得多的规模上训练, 并在诸如 GLUE (Wang et al., 2019) 和 SuperGLUE (Wang et al., 2020) 等基准上表现得极具竞争力。我们评估 LoRA 是否仍能在 GLUE 上达到完全微调的 DeBERTa XXL (1.5B) 的性能。结果呈现在表 2

(最下部分) 中。所用超参数的细节见 D.2 节。

5.4 GPT-2 medium/large

在表明 LoRA 在 NLU 上可以成为完全微调的有竞争力替代方案之后, 我们希望回答 LoRA 在 NLG 模型 (如 GPT-2 medium 和 large (Radford et al., b)) 上是否仍占优势。为进行直接比较, 我们使我们的设置尽可能贴近 Li & Liang (2021)。由于篇幅限制, 本节中我们只给出在 E2E NLG Challenge 上的结果 (表 3)。在 WebNLG (Gardent et al., 2017) 和 DART (Nan et al., 2020) 上的结果见 F.1 节。我们在 D.3 节中列出了所用的超参数。

表 4: 在 GPT-3 175B 上不同自适应方法的性能。我们报告 WikiSQL 上的逻辑形式 (logical form) 验证准确率、MultiNLI-matched 上的验证准确率, 以及 SAMSum 上的 Rouge-1/2/L。LoRA 的表现优于先前的方法, 包括完全微调。WikiSQL 上的结果波动约为 $\pm 0.5\%$, MNLI-m 约为 $\pm 0.1\%$, SAMSum 三项指标分别约为 $\pm 0.2/\pm 0.2/\pm 0.1$ 。

模型与方法	参数量	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

5.5 扩展到 GPT-3 175B (Scaling up to GPT-3 175B)

作为对 LoRA 的最终压力测试, 我们将规模扩展到拥有 1750 亿参数的 GPT-3。由于训练成本高昂, 我们只报告给定任务在不同随机种子上的典型标准差, 而非为每个条目都给出一个标准差。所用超参数的细节见 D.4 节。如表 4 所示, LoRA 在所有三个数据集上都达到或超过了微调基线。注意, 并非所有方法都能从更多可训练参数中单调获益, 如图 2 所示。对于前缀嵌入微调, 当我们使用超过 256 个特殊 token 时; 或对于前缀层微调, 当我们使用超过 32 个特殊 token 时, 我们都观察到性能显著下降。这印证了 Li & Liang (2021) 中类似的观察结果。虽然对这一现象进行透彻研究超出了本工作的范围, 但我们怀疑使用更多的特殊 token 会导致输入分布进一步偏离预训练数据分布。另外, 我们在 F.3 节中研究了不同自适应方法在低数据量场景 (low-data regime) 下的性能。

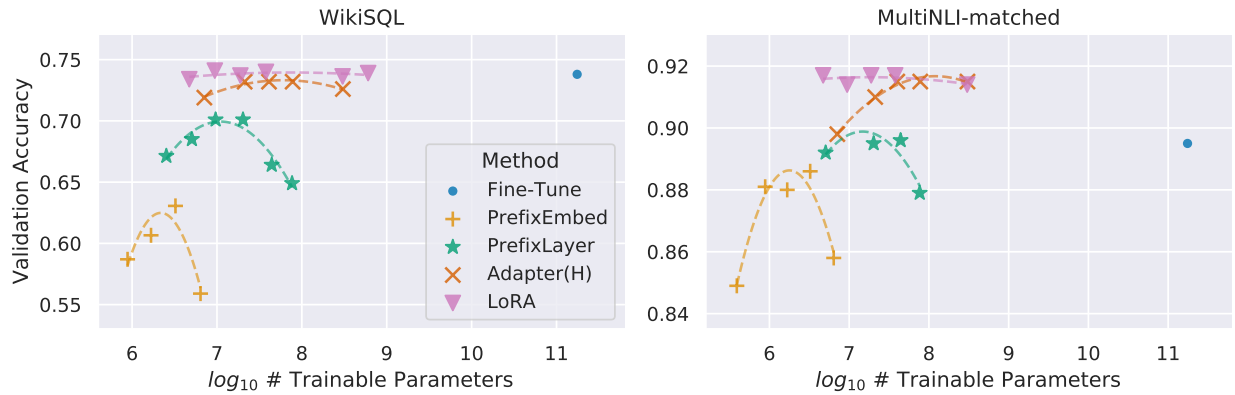


图 2: 在 WikiSQL 和 MNLI-matched 上, 若干自适应方法的 GPT-3 175B 验证准确率与可训练参数数量的关系。LoRA 展现出更好的可扩展性 (scalability) 和任务性能。所绘数据点的更多细节见 F.2 节。

6 相关工作 (Related Works)

Transformer 语言模型 Transformer (Vaswani et al., 2017) 是一种大量使用自注意力的序列到序列 (sequence-to-sequence) 架构。Radford et al. (a) 通过堆叠 Transformer 解码器 (decoder) 将其应用于自回归语言建模。从那时起, 基于 Transformer 的语言模型在 NLP 领域占据主导地位, 在许多任务上取得了最先进的水平。一种新范式随 BERT (Devlin et al., 2019b) 和 GPT-2 (Radford et al., b) 的出现而兴起——二者都是在大量文本上训练的大型 Transformer 语言模型——其中, 在通用领域数据上预训练之后再在任务专属数据上微调, 相比直接在任务专属数据上训练能带来显著的性能提升。训练更大的 Transformer 通常会带来更好的性能, 并且仍是一个活跃的研究方向。GPT-3 (Brown et al., 2020) 是迄今为止训练出的最大的单一 Transformer 语言模型, 拥有 175B 参数。

提示工程与微调 (Prompt Engineering and Fine-Tuning) 尽管 GPT-3 175B 仅需少量额外训练样本就能调整其行为, 但其结果在很大程度上依赖于输入提示 (Brown et al., 2020)。这就需要一门关于撰写和组织提示格式的实证技艺, 以最大化模型在所需任务上的性能, 这被称为提示工程 (prompt engineering) 或提示黑客 (prompt hacking)。微调将一个在通用领域上预训练的模型重新训练到特定任务 (Devlin et al., 2019b; Radford et al., a)。它的变体包括只学习参数的一个子集 (Devlin et al., 2019b; Collobert & Weston, 2008), 但从业者往往会重新训练全部参数以最大化下游性能。然而, GPT-3 175B 的庞大规模使得以通常方式进行微调变得困难, 因为它会产生巨大的检查点, 且硬件准入门槛很高——其内存占用与预训练相同。

参数高效的自适应 (Parameter-Efficient Adaptation) 许多人提出在神经网络的现有层之间插入 adapter 层 (Houlsby et al., 2019; Rebuffi et al., 2017; Lin et al., 2020)。我们的方法使用类似的瓶颈结构来对权重更新施加低秩约束。关键的功能差异在于, 我们学习到的权重可以在推理时与主权重合并, 从而不引入任何延迟, 而 adapter 层则做不到这一点 (第 3 节)。adapter 的一个同期扩展是 COMPACTER (Mahabadi et al., 2021), 它本质上使用 Kronecker 积 (Kronecker products) 配合某种

预先确定的权重共享方案来参数化 adapter 层。类似地, 将 LoRA 与其他基于张量积 (tensor product) 的方法相结合可能会进一步提升其参数效率, 我们将其留待未来工作。最近, 许多人提出优化输入词嵌入以替代微调, 这类似于提示工程的一种连续且可微的推广 (Li & Liang, 2021; Lester et al., 2021; Hambarzumyan et al., 2020; Liu et al., 2021)。我们在实验部分纳入了与 Li & Liang (2021) 的比较。然而, 这一系列工作只能通过在提示中使用更多特殊 token 来扩展规模, 而当位置嵌入 (positional embeddings) 被学习时, 这些 token 会占用本可用于任务 token 的序列长度。

深度学习中的低秩结构 (Low-Rank Structures in Deep Learning) 低秩结构在机器学习中非常常见。许多机器学习问题都具有某种内在的低秩结构 (Li et al., 2016; Cai et al., 2010; Li et al., 2018b; Grasedyck et al., 2013)。此外, 众所周知, 对于许多深度学习任务, 尤其是那些使用高度过参数化神经网络的任务, 学习得到的神经网络在训练后会具有低秩性质 (Oymak et al., 2019)。一些先前的工作甚至在训练原始神经网络时就显式地施加低秩约束 (Sainath et al., 2013; Povey et al., 2018; Zhang et al., 2014; Jaderberg et al., 2014; Zhao et al., 2016; Khodak et al., 2021; Denil et al., 2014); 然而, 据我们所知, 这些工作中没有一个考虑对一个冻结模型进行低秩更新以自适应到下游任务。在理论文献中, 已知当底层概念类 (concept class) 具有某种低秩结构时 (Ghorbani et al., 2020; Allen-Zhu & Li, 2019; Allen-Zhu & Li, 2020a), 神经网络优于其他经典学习方法, 包括相应的 (有限宽度) 神经正切核 (neural tangent kernels) (Allen-Zhu et al., 2019; Li & Liang, 2018)。Allen-Zhu & Li (2020b) 中的另一个理论结果表明, 低秩自适应对抗训练 (adversarial training) 可能是有用的。总之, 我们相信我们提出的低秩自适应更新有充分的文献依据支撑。

7 理解低秩更新 (Understanding the Low-Rank Updates)

鉴于 LoRA 的实证优势, 我们希望进一步解释从下游任务中学习到的低秩自适应的性质。注意, 低秩结构不仅降低了硬件准入门槛——这使我们能够并行运行多个实验——而且还为更新权重如何与预训练权重相关联提供了更好的可解释性。我们将研究聚焦于 GPT-3 175B, 在该模型上, 我们在不对任务性能产生不利影响的情况下, 实现了可训练参数最大幅度的减少 (最多 10,000 倍)。

我们进行了一系列实证研究, 以回答以下问题: 1) 在给定参数预算约束的情况下, 我们应该自适应预训练 Transformer 中的哪个权重矩阵子集, 才能最大化下游性能? 2) “最优”的自适应矩阵 ΔW 真的秩亏吗? 如果是, 在实践中使用多大的秩比较好? 3) ΔW 与 W 之间有什么联系? ΔW 与 W 高度相关吗? ΔW 相比 W 有多大?

我们相信, 我们对问题 (2) 和 (3) 的回答揭示了将预训练语言模型用于下游任务的基本原理, 这是 NLP 中的一个关键课题。

7.1 我们应该将 LoRA 应用于 Transformer 中的哪些权重矩阵？(Which Weight Matrices in Transformer Should We Apply LoRA to?)

在给定有限参数预算的情况下，我们应该用 LoRA 自适应哪些类型的权重，才能在下游任务上获得最佳性能？如 4.2 节所述，我们只考虑自注意力模块中的权重矩阵。我们在 GPT-3 175B 上设定 18M 的参数预算（若以 FP16 存储约为 35MB），对于全部 96 层而言，这对应于：若我们自适应一种类型的注意力权重则 $r = 8$ ，若自适应两种类型则 $r = 4$ 。结果呈现在表 5 中。

表 5: 在可训练参数数量相同的情况下，将 LoRA 应用于 GPT-3 中不同类型注意力权重后在 WikiSQL 和 MultiNLI 上的验证准确率（可训练参数数量 = 18M）。同时自适应 W_q 和 W_v 总体上给出最佳性能。我们发现给定数据集在不同随机种子间的标准差是一致的，我们在第一列中报告。

权重类型	W_q	W_k	W_v	W_o	W_q, W_k	W_q, W_v	W_q, W_k, W_v, W_o
秩 r	8	8	8	8	4	4	2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

注意，将所有参数都用于 ΔW_q 或 ΔW_k 会导致性能显著降低，而同时自适应 W_q 和 W_v 则产生最佳结果。这表明，即便秩为四也能捕获 ΔW 中足够的信息，以至于自适应更多权重矩阵比用更大的秩自适应单一类型的权重更可取。

7.2 LoRA 的最优秩 r 是多少？(What is the Optimal Rank r for LoRA?)

我们将注意力转向秩 r 对模型性能的影响。我们自适应 $\{W_q, W_v\}$ 、 $\{W_q, W_k, W_v, W_o\}$ 以及仅 W_q 来进行比较。

表 6: 在不同秩 r 下，WikiSQL 和 MultiNLI 上的验证准确率。令我们惊讶的是，在这些数据集上，小到为一的秩就足以同时自适应 W_q 和 W_v ，而单独训练 W_q 则需要更大的 r 。我们在 H.2 节中对 GPT-2 进行了类似的实验。

	权重类型	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL ($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

表 6 表明, 令人惊讶的是, LoRA 在非常小的 r 下就已表现出竞争力 (对 $\{W_q, W_v\}$ 比仅对 W_q 更为明显)。这表明更新矩阵 ΔW 可能具有非常小的“内在秩”。⁶为进一步支持这一发现, 我们检查了由不同 r 的选择以及由不同随机种子所学习到的子空间之间的重叠。我们论证: 增大 r 并不会覆盖一个更有意义的子空间, 这表明一个低秩自适应矩阵就足够了。

不同 r 之间的子空间相似度 给定 $A_{r=8}$ 和 $A_{r=64}$ (它们是使用相同预训练模型、以秩 $r = 8$ 和 64 学习得到的自适应矩阵), 我们进行奇异值分解 (singular value decomposition), 并得到右奇异酉矩阵 (right-singular unitary matrices) $U_{A_{r=8}}$ 和 $U_{A_{r=64}}$ 。⁷我们希望回答: $U_{A_{r=8}}$ 中前 i 个奇异向量所张成的子空间 ($1 \leq i \leq 8$), 有多少被包含在 $U_{A_{r=64}}$ 中前 j 个奇异向量所张成的子空间 ($1 \leq j \leq 64$) 之内? 我们用一个基于 Grassmann 距离的归一化子空间相似度来度量这一量 (更正式的讨论见附录 G):

$$\phi(A_{r=8}, A_{r=64}, i, j) = \frac{\|U_{A_{r=8}}^{i\top} U_{A_{r=64}}^j\|_F^2}{\min(i, j)} \in [0, 1] \quad (4)$$

其中 $U_{A_{r=8}}^i$ 表示 $U_{A_{r=8}}$ 中对应于前 i 个奇异向量的列。 $\phi(\cdot)$ 的取值范围为 $[0, 1]$, 其中 1 表示子空间完全重叠, 0 表示完全分离。 ϕ 随 i 和 j 变化的情况见图 3。由于篇幅限制, 我们只看第 48 层 (共 96 层), 但该结论对其他层同样成立, 如 H.1 节所示。

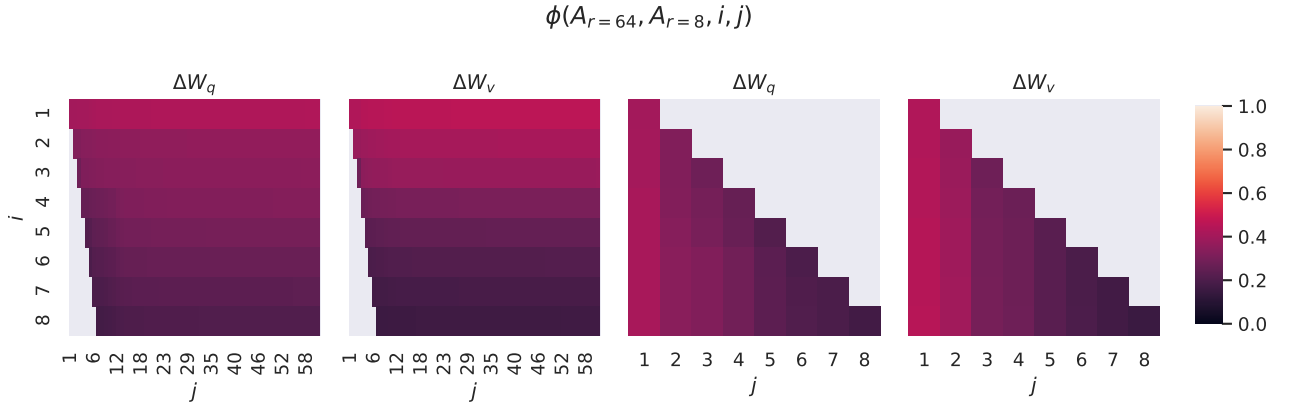


图 3: 对于 ΔW_q 和 ΔW_v , $A_{r=8}$ 与 $A_{r=64}$ 列向量之间的子空间相似度。第三和第四幅图放大了前两幅图中左下角的三角形区域。 $r = 8$ 中的顶部方向被包含在 $r = 64$ 中, 反之亦然。

我们从图 3 中得出一个重要观察。

对应于顶部奇异向量的方向在 $A_{r=8}$ 与 $A_{r=64}$ 之间显著重叠, 而其他方向则不然。具体而言, $A_{r=8}$ 的 ΔW_v (相应地 ΔW_q) 与 $A_{r=64}$ 的 ΔW_v (相应地 ΔW_q) 共享一个维度为 1、归一化相似度 > 0.5 的子空间, 这解释了为何 $r = 1$ 在我们针对 GPT-3 的下游任务中表现相当不错。

⁶然而, 我们并不期望小的 r 对每个任务或数据集都有效。考虑以下思想实验: 如果下游任务所使用的语言与预训练所用的语言不同, 那么重新训练整个模型 (类似于 $r = d_{\text{model}}$ 的 LoRA) 肯定能胜过使用小 r 的 LoRA。

⁷注意, 类似的分析也可以用 B 和左奇异酉矩阵来进行——在我们的实验中, 我们坚持使用 A 。

由于 $A_{r=8}$ 和 $A_{r=64}$ 都是使用相同的预训练模型学习得到的，图 3 表明 $A_{r=8}$ 和 $A_{r=64}$ 的顶部奇异向量方向是最有用的，而其他方向可能主要包含训练期间累积的随机噪声。因此，自适应矩阵确实可以具有非常低的秩。

不同随机种子之间的子空间相似度 我们通过绘制两个采用 $r = 64$ 、随机种子不同的运行之间的归一化子空间相似度（如图 4 所示）进一步证实了这一点。 ΔW_q 似乎比 ΔW_v 具有更高的“内在秩”，因为对于 ΔW_q ，两次运行都学习到了更多共同的奇异值方向，这与我们在表 6 中的实证观察相符。作为对比，我们还绘制了两个随机高斯矩阵，它们彼此之间不共享任何共同的奇异值方向。

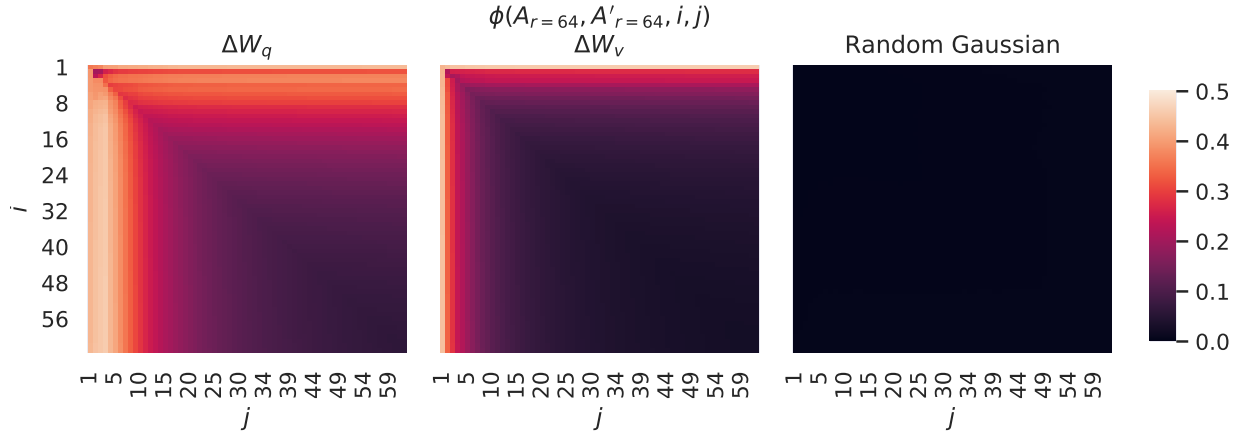


图 4: 左图与中图：在 ΔW_q （左）和 ΔW_v （中）上，两个采用 $r = 64$ 、随机种子不同的运行之间所学习到的归一化子空间相似度。右图：两个随机高斯矩阵之间的同一热力图。其他层见图 7。

7.3 自适应矩阵 ΔW 与 W 相比如何？(How Does the Adaptation Matrix ΔW Compare to W ?)

我们进一步研究 ΔW 与 W 之间的关系。特别地， ΔW 与 W 高度相关吗？（或者用数学语言说， ΔW 是否大部分包含在 W 的顶部奇异方向之中？）此外， ΔW 相比其在 W 中对应的方向有多“大”？这能够揭示自适应预训练语言模型的底层机制。

为回答这些问题，我们将 W 投影到 ΔW 的 r 维子空间上，方法是计算 $U^\top W V^\top$ ，其中 U/V 是 ΔW 的左/右奇异向量矩阵。然后，我们比较 $\|U^\top W V^\top\|_F$ 与 $\|W\|_F$ 之间的 Frobenius 范数。作为对比，我们还将 U, V 替换为 W 的前 r 个奇异向量或一个随机矩阵的奇异向量，来计算 $\|U^\top W V^\top\|_F$ 。

表 7: $U^\top W_q V^\top$ 的 Frobenius 范数, 其中 U 和 V 是以下三者之一的左/右前 r 个奇异向量方向: (1) ΔW_q , (2) W_q , 或 (3) 一个随机矩阵。这些权重矩阵取自 GPT-3 的第 48 层。

	$r = 4$			$r = 64$		
	ΔW_q	W_q	Random	ΔW_q	W_q	Random
$\ U^\top W_q V^\top\ _F =$	0.32	21.67	0.02	1.90	37.71	0.33

其中 $\|W_q\|_F = 61.95$; 对 $r = 4$, $\|\Delta W_q\|_F = 6.91$; 对 $r = 64$, $\|\Delta W_q\|_F = 3.57$ 。

我们从表 7 中得出若干结论。第一, 与随机矩阵相比, ΔW 与 W 具有更强的相关性, 这表明 ΔW 放大了 W 中已有的某些特征。第二, ΔW 并非重复 W 的顶部奇异方向, 而是只放大那些在 W 中未被强调的方向。第三, 放大因子相当大: 对 $r = 4$, $21.5 \approx 6.91/0.32$ 。关于为何 $r = 64$ 具有较小的放大因子, 见 H.4 节。我们还在 H.3 节中提供了一个可视化, 展示当我们纳入 W_q 中更多顶部奇异方向时相关性如何变化。这表明, 低秩自适应矩阵可能放大了那些在通用预训练模型中已被学习但未被强调的、对特定下游任务重要的特征。

8 结论与未来工作 (Conclusion and Future Work)

就所需硬件以及为不同任务托管独立实例的存储/切换成本而言, 对庞大的语言模型进行微调代价高昂得令人望而却步。我们提出了 LoRA, 一种高效的自适应策略, 它既不引入推理延迟, 也不减少输入序列长度, 同时保持了高模型质量。重要的是, 当作为服务部署时, 它通过共享绝大多数模型参数实现了快速的任务切换。虽然我们关注的是 Transformer 语言模型, 但所提出的原理普遍适用于任何带有稠密层的神经网络。

未来工作有许多方向。1) LoRA 可以与其他高效自适应方法相结合, 可能提供正交的改进。2) 微调或 LoRA 背后的机制远未明朗——在预训练期间学习到的特征是如何被转化以在下游任务上表现良好的? 我们相信, 相比完全微调, LoRA 使回答这个问题变得更易处理。3) 我们主要依赖启发式方法来选择应用 LoRA 的权重矩阵。是否有更有原则的方法来做这件事? 4) 最后, ΔW 的秩亏暗示 W 本身也可能是秩亏的, 这同样可以成为未来工作的灵感来源。

参考文献 (References)

1. Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. arXiv:2012.13255 [cs], December 2020. URL <http://arxiv.org/abs/2012.13255>.
2. Zeyuan Allen-Zhu and Yuanzhi Li. What Can ResNet Learn Efficiently, Going Beyond Kernels? In NeurIPS, 2019. Full version available at <http://arxiv.org/abs/1905.10337>.

3. Zeyuan Allen-Zhu and Yuanzhi Li. Backward feature correction: How deep learning performs deep learning. arXiv preprint arXiv:2001.04413, 2020a.
4. Zeyuan Allen-Zhu and Yuanzhi Li. Feature purification: How adversarial training performs robust deep learning. arXiv preprint arXiv:2005.10190, 2020b.
5. Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. In ICML, 2019. Full version available at <http://arxiv.org/abs/1811.03962>.
6. Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.
7. Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs], July 2020. URL <http://arxiv.org/abs/2005.14165>.
8. Jian-Feng Cai, Emmanuel J Candès, and Zuowei Shen. A singular value thresholding algorithm for matrix completion. SIAM Journal on optimization, 20(4):1956–1982, 2010.
9. Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017), 2017. doi: 10.18653/v1/s17-2001. URL <http://dx.doi.org/10.18653/v1/S17-2001>.
10. Ronan Collobert and Jason Weston. A unified architecture for natural language processing: deep neural networks with multitask learning. In Proceedings of the 25th international conference on Machine learning, ICML '08, pp. 160–167, New York, NY, USA, July 2008. Association for Computing Machinery. ISBN 978-1-60558-205-4. doi: 10.1145/1390156.1390177. URL <https://doi.org/10.1145/1390156.1390177>.
11. Misha Denil, Babak Shakibi, Laurent Dinh, Marc’Aurelio Ranzato, and Nando de Freitas. Predicting parameters in deep learning, 2014.
12. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019a.
13. Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs], May 2019b. URL <http://arxiv.org/abs/1810.04805>. arXiv: 1810.04805.
14. William B. Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In Proceedings of the Third International Workshop on Paraphrasing (IWP2005), 2005.

URL <https://aclanthology.org/I05-5002>.

15. Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. The webnlg challenge: Generating text from rdf data. In *Proceedings of the 10th International Conference on Natural Language Generation*, pp. 124–133, 2017.
16. Behrooz Ghorbani, Song Mei, Theodor Misiakiewicz, and Andrea Montanari. When do neural networks outperform kernel methods? *arXiv preprint arXiv:2006.13409*, 2020.
17. Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. Samsun corpus: A human-annotated dialogue dataset for abstractive summarization. *CoRR*, abs/1911.12237, 2019. URL <http://arxiv.org/abs/1911.12237>.
18. Lars Grasedyck, Daniel Kressner, and Christine Tobler. A literature survey of low-rank tensor approximation techniques. *GAMM-Mitteilungen*, 36(1):53–78, 2013.
19. Jihun Ham and Daniel D. Lee. Grassmann discriminant analysis: a unifying view on subspace-based learning. In *ICML*, pp. 376–383, 2008. URL <https://doi.org/10.1145/1390156.1390204>.
20. Karen Hambardzumyan, Hrant Khachatrian, and Jonathan May. WARP: Word-level Adversarial ReProgramming. *arXiv:2101.00121 [cs]*, December 2020. URL <http://arxiv.org/abs/2101.00121>. arXiv: 2101.00121.
21. Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention, 2021.
22. Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. *arXiv:1902.00751 [cs, stat]*, June 2019. URL <http://arxiv.org/abs/1902.00751>.
23. Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. Speeding up convolutional neural networks with low rank expansions. *arXiv preprint arXiv:1405.3866*, 2014.
24. Mikhail Khodak, Neil Tenenholtz, Lester Mackey, and Nicolò Fusi. Initialization and regularization of factorized neural layers, 2021.
25. Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
26. Dmitry Lepikhin, HyukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.
27. Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. *arXiv:2104.08691 [cs]*, April 2021. URL <http://arxiv.org/abs/2104.08691>. arXiv: 2104.08691.

28. Chunyuan Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. Measuring the Intrinsic Dimension of Objective Landscapes. arXiv:1804.08838 [cs, stat], April 2018a. URL <http://arxiv.org/abs/1804.08838>. arXiv: 1804.08838.
29. Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. arXiv:2101.00190 [cs], January 2021. URL <http://arxiv.org/abs/2101.00190>.
30. Yuanzhi Li and Yingyu Liang. Learning overparameterized neural networks via stochastic gradient descent on structured data. In Advances in Neural Information Processing Systems, 2018.
31. Yuanzhi Li, Yingyu Liang, and Andrej Risteski. Recovery guarantee of weighted low-rank approximation via alternating minimization. In International Conference on Machine Learning, pp. 2358–2367. PMLR, 2016.
32. Yuanzhi Li, Tengyu Ma, and Hongyang Zhang. Algorithmic regularization in over-parameterized matrix sensing and neural networks with quadratic activations. In Conference On Learning Theory, pp. 2–47. PMLR, 2018b.
33. Zhaojiang Lin, Andrea Madotto, and Pascale Fung. Exploring versatile generative language model via parameter-efficient transfer learning. In Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 441–459, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.41. URL <https://aclanthology.org/2020.findings-emnlp.41>.
34. Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. GPT Understands, Too. arXiv:2103.10385 [cs], March 2021. URL <http://arxiv.org/abs/2103.10385>. arXiv: 2103.10385.
35. Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019.
36. Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101, 2017.
37. Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.
38. Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers, 2021.
39. Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, et al. Dart: Open-domain structured data record to text generation. arXiv preprint arXiv:2007.02871, 2020.
40. Jekaterina Novikova, Ondřej Dušek, and Verena Rieser. The e2e dataset: New challenges for end-to-end generation. arXiv preprint arXiv:1706.09254, 2017.

41. Samet Oymak, Zalan Fabian, Mingchen Li, and Mahdi Soltanolkotabi. Generalization guarantees for neural networks via harnessing the low-rank structure of the jacobian. arXiv preprint arXiv:1906.05392, 2019.
42. Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning, 2021.
43. Daniel Povey, Gaofeng Cheng, Yiming Wang, Ke Li, Hainan Xu, Mahsa Yarmohammadi, and Sanjeev Khudanpur. Semi-orthogonal low-rank matrix factorization for deep neural networks. In Interspeech, pp. 3743–3747, 2018.
44. Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving Language Understanding by Generative Pre-Training. pp. 12, a.
45. Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. pp. 24, b.
46. Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. CoRR, abs/1806.03822, 2018. URL <http://arxiv.org/abs/1806.03822>.
47. Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Learning multiple visual domains with residual adapters. arXiv:1705.08045 [cs, stat], November 2017. URL <http://arxiv.org/abs/1705.08045>. arXiv: 1705.08045.
48. Andreas Rücklé, Gregor Geigle, Max Glockner, Tilman Beck, Jonas Pfeiffer, Nils Reimers, and Iryna Gurevych. Adapterdrop: On the efficiency of adapters in transformers, 2020.
49. Tara N Sainath, Brian Kingsbury, Vikas Sindhwani, Ebru Arisoy, and Bhuvana Ramabhadran. Low-rank matrix factorization for deep neural network training with high-dimensional output targets. In 2013 IEEE international conference on acoustics, speech and signal processing, pp. 6655–6659. IEEE, 2013.
50. Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.
51. Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing, pp. 1631–1642, Seattle, Washington, USA, October 2013. Association for Computational Linguistics. URL <https://aclanthology.org/D13-1170>.
52. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems, pp. 6000–6010, 2017.

53. Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.
54. Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems, 2020.
55. Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Neural network acceptability judgments. arXiv preprint arXiv:1805.12471, 2018.
56. Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), pp. 1112–1122, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1101. URL <https://www.aclweb.org/anthology/N18-1101>.
57. Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
58. Greg Yang and Edward J. Hu. Feature Learning in Infinite-Width Neural Networks. arXiv:2011.14522 [cond-mat], May 2021. URL <http://arxiv.org/abs/2011.14522>. arXiv: 2011.14522.
59. Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models, 2021.
60. Yu Zhang, Ekapol Chuangsuwanich, and James Glass. Extracting deep neural network bottleneck features using low-rank matrix factorization. In 2014 IEEE international conference on acoustics, speech and signal processing (ICASSP), pp. 185–189. IEEE, 2014.
61. Yong Zhao, Jinyu Li, and Yifan Gong. Low-rank plus diagonal adaptation for deep neural networks. In 2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 5005–5009. IEEE, 2016.
62. Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. CoRR, abs/1709.00103, 2017. URL <http://arxiv.org/abs/1709.00103>.

A 大语言模型仍然需要参数更新 (Large Language Models Still Need Parameter Updates)

当我们只有少量训练样本时，少样本学习 (few-shot learning) 或提示工程非常有利。然而在实践中，对于性能敏感的应用，我们往往能够负担得起整理数千个甚至更多的训练样例。如表 8 所示，在大大小小的数据集上，微调相比少样本学习都能极大地改善模型性能。我们从 GPT-3 论文 (Brown et al., 2020) 中取得 GPT-3 在 RTE 上的少样本结果。对于 MNLI-matched，我们为每个类别使用两个示范 (demonstration)，共计六个上下文内 (in-context) 样例。

表 8: 在 GPT-3 上，微调显著优于少样本学习 (Brown et al., 2020)。

方法	MNLI-m (验证准确率/%)	RTE (验证准确率/%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

B Adapter 层引入的推理延迟 (Inference Latency Introduced by Adapter Layers)

adapter 层是以顺序方式添加到预训练模型上的外部模块，而我们的提案 LoRA 则可以视为以并行方式添加的外部模块。因此，adapter 层必须在基础模型之外被额外计算，不可避免地引入额外延迟。正如 Rücklé et al. (2020) 所指出的，当模型批大小和/或序列长度足够大、能充分利用硬件并行性时，adapter 层引入的延迟可以得到缓解。我们通过在 GPT-2 medium 上进行的一项类似延迟研究证实了他们的观察，并指出存在一些场景——尤其是批大小较小的在线推理——在这些场景中，所增加的延迟可能十分显著。

我们在一块 NVIDIA Quadro RTX8000 上测量单次前向传递的延迟，对 100 次试验取平均。我们改变输入批大小、序列长度以及 adapter 瓶颈维度 r 。我们测试两种 adapter 设计：Houlsby et al. (2019) 的原始设计 (我们称之为 Adapter^H)，以及 Lin et al. (2020) 最近一种更高效的变体 (我们称之为 Adapter^L)。关于这些设计的更多细节见 5.1 节。我们在图 5 中以百分比形式绘制相对于无 adapter 基线的减速 (slow-down)。

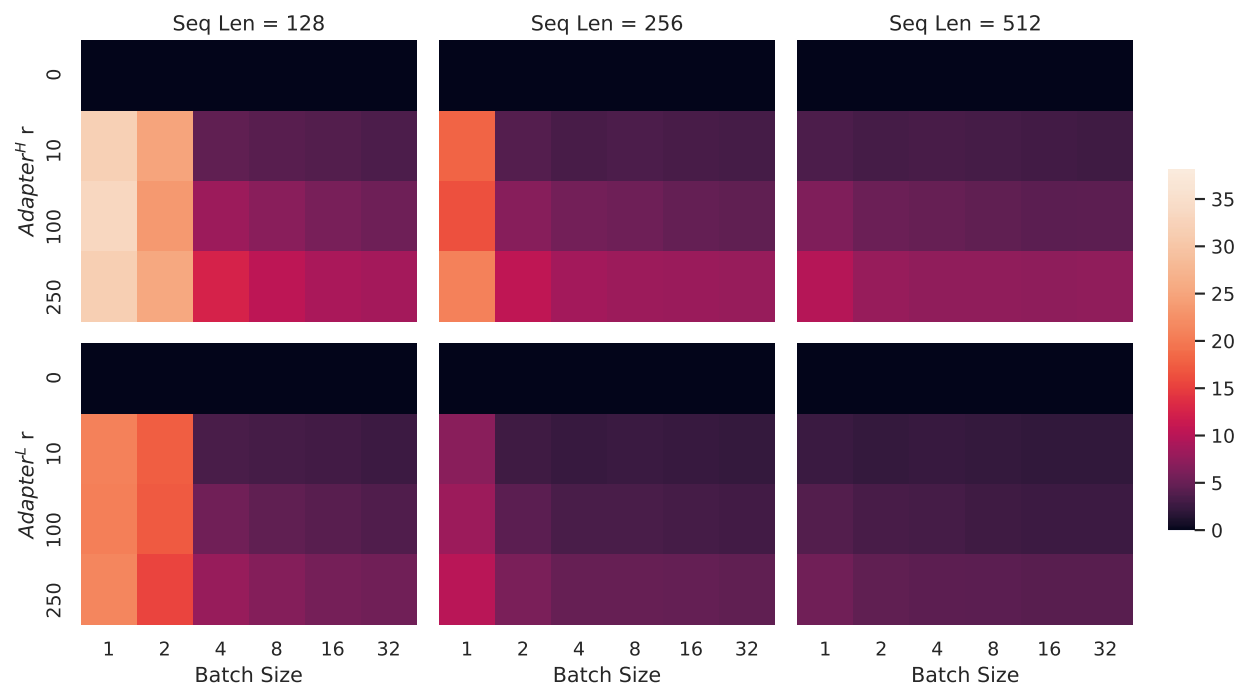


图 5: 推理延迟相对于无 adapter ($r = 0$) 基线的减速百分比。上排展示 Adapter^H 的结果, 下排展示 Adapter^L 。更大的批大小和序列长度有助于缓解延迟, 但在在线、短序列长度的场景中, 减速可能高达 30% 以上。我们调整了配色映射以提高可见度。

C 数据集细节 (Dataset Details)

GLUE 基准 (GLUE Benchmark) 是一个范围广泛的自然语言理解任务集合。它包括 MNLI (推理, Williams et al. (2018))、SST-2 (情感分析, Socher et al. (2013))、MRPC (释义检测, Dolan & Brockett (2005))、CoLA (语言可接受性, Warstadt et al. (2018))、QNLI (推理, Rajpurkar et al. (2018))、QQP⁸ (问答)、RTE (推理), 以及 STS-B (文本相似度, Cer et al. (2017))。其广泛的覆盖面使 GLUE 基准成为评估 RoBERTa、DeBERTa 等 NLU 模型的标准指标。各个数据集以不同的宽松许可证发布。

WikiSQL 在 Zhong et al. (2017) 中引入, 包含 56,355/8,421 个训练/验证样例。任务是从自然语言问题和表模式 (table schema) 生成 SQL 查询。我们将上下文编码为 $x = \{\text{表模式, 查询}\}$, 将目标编码为 $y = \{\text{SQL}\}$ 。该数据集以 BSD 3-Clause License 发布。

SAMSum 在 Gliwa et al. (2019) 中引入, 包含 14,732/819 个训练/测试样例。它由两人之间编排的聊天对话以及由语言学家撰写的相应抽象式摘要 (abstractive summaries) 组成。我们将上下文编码为用 “\n” 连接、其后跟一个 “\n\n” 的话语序列, 将目标编码为 $y = \{\text{摘要}\}$ 。该数据集以非商业许可证 Creative Commons BY-NC-ND 4.0 发布。

⁸<https://quoradata.quora.com/First-Quora-Dataset-Release-Question-Pairs>

E2E NLG Challenge 最初在 Novikova et al. (2017) 中引入，作为一个用于训练端到端、数据驱动的自然语言生成系统的数据集，常用于数据到文本 (data-to-text) 的评估。E2E 数据集由来自餐厅领域的大约 42,000 个训练、4,600 个验证和 4,600 个测试样例组成。作为输入使用的每个源表可以有多个参考。每个样本输入 (x, y) 由一个槽值对 (slot-value pairs) 序列以及一段相应的自然语言参考文本组成。该数据集以 Creative Commons BY-NC-SA 4.0 发布。

DART 是 Nan et al. (2020) 中描述的一个开放领域数据到文本数据集。DART 的输入被组织为“实体 (ENTITY) — 关系 (RELATION) — 实体 (ENTITY)”三元组的序列。DART 总共有 82K 个样例，与 E2E 相比，是一个规模大得多、复杂得多的数据到文本任务。该数据集以 MIT 许可证发布。

WebNLG 是另一个常用于数据到文本评估的数据集 (Gardent et al., 2017)。WebNLG 总共有 22K 个样例，包含 14 个不同类别，其中九个在训练期间可见。由于全部 14 个类别中有五个在训练期间不可见、但在测试集中有所体现，因此评估通常被拆分为“可见”类别 (S)、“不可见”类别 (U) 和“全部” (A)。每个输入样例由一个“主语 (SUBJECT) — 属性 (PROPERTY) — 宾语 (OBJECT)”三元组序列表示。该数据集以 Creative Commons BY-NC-SA 4.0 发布。

D 实验中所用的超参数 (Hyperparameters Used in Experiments)

D.1 RoBERTa

我们使用 AdamW 配合线性学习率衰减 (linear learning rate decay) 调度进行训练。我们为 LoRA 扫描 (sweep) 学习率、训练轮数 (epochs) 和批大小。遵循 Liu et al. (2019)，在自适应到 MRPC、RTE 和 STS-B 时，我们将 LoRA 模块初始化为我们最佳的 MNLI 检查点，而非通常的初始化；对所有任务，预训练模型保持冻结。我们报告 5 个随机种子的中位数；每次运行的结果取自最佳轮。为了与 Houlsby et al. (2019) 和 Pfeiffer et al. (2021) 的设置公平比较，我们将模型序列长度限制为 128，并对所有任务使用固定的批大小。重要的是，在自适应到 MRPC、RTE 和 STS-B 时，我们从预训练的 RoBERTa large 模型开始，而非一个已经自适应到 MNLI 的模型。采用这一受限设置的运行用 $\dagger$ 标记。我们运行中所用的超参数见表 9。

D.2 DeBERTa

我们再次使用 AdamW 配合线性学习率衰减调度进行训练。遵循 He et al. (2021)，我们调整学习率、丢弃 (dropout) 概率、预热步数 (warm-up steps) 和批大小。为保持比较公平，我们使用与 (He et al., 2021) 相同的模型序列长度。遵循 He et al. (2021)，在自适应到 MRPC、RTE 和 STS-B 时，我们将 LoRA 模块初始化为我们最佳的 MNLI 检查点，而非通常的初始化；对所有任务，预训练模型保持冻结。我们报告 5 个随机种子的中位数；每次运行的结果取自最佳轮。我们运行中所用的超参数见表 10。

D.3 GPT-2

我们使用 AdamW (Loshchilov & Hutter, 2017) 配合线性学习率调度, 对我们所有的 GPT-2 模型训练 5 轮。我们使用 Li & Liang (2021) 中描述的批大小、学习率和束搜索 (beam search) 束宽。相应地, 我们也为 LoRA 调整上述超参数。我们报告 3 个随机种子的均值; 每次运行的结果取自最佳轮。GPT-2 中 LoRA 所用的超参数列于表 11。对于其他基线所用的超参数, 见 Li & Liang (2021)。

D.4 GPT-3

对于所有 GPT-3 实验, 我们使用 AdamW (Loshchilov & Hutter, 2017) 训练 2 轮, 批大小为 128 个样本, 权重衰减 (weight decay) 因子为 0.1。我们对 WikiSQL (Zhong et al., 2017) 使用 384 的序列长度, 对 MNLI (Williams et al., 2018) 使用 768, 对 SAMSum (Gliwa et al., 2019) 使用 2048。我们为所有方法-数据集组合调整学习率。所用超参数的更多细节见 D.4 节。对于前缀嵌入微调, 我们发现最优的 l_p 和 l_i 分别为 256 和 8, 共计 3.2M 个可训练参数。对于前缀层微调, 我们使用 $l_p = 8$ 和 $l_i = 8$ 、共 20.2M 个可训练参数, 以获得总体最佳性能。我们为 LoRA 给出两种参数预算: 4.7M ($r_q = r_v = 1$ 或 $r_v = 2$) 和 37.7M ($r_q = r_v = 8$ 或 $r_q = r_k = r_v = r_o = 2$)。我们报告每次运行的最佳验证性能。我们 GPT-3 实验中所用的训练超参数列于表 12。

表 10: GLUE 基准所含任务上 DeBERTa XXL 的超参数。

方法	数据集	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B
DeBERTa XXL LoRA	Optimizer	AdamW							
	Warmup Ratio	0.1							
	LR Schedule	Linear							
	Batch Size	8	8	32	4	6	8	4	4
	# Epochs	5	16	30	10	8	11	11	10
	Learning Rate	1E-04	6E-05	2E-04	1E-04	1E-04	1E-04	2E-04	2E-04
	Weight Decay	0	0.01	0.01	0	0.01	0.01	0.01	0.1
	CLS Dropout	0.15	0	0	0.1	0.1	0.2	0.2	0.2
	LoRA Config.	$r_q = r_v = 8$							
	LoRA α	8							
	Max Seq. Len.	256	128	128	64	512	320	320	128

表 11: GPT-2 LoRA 在 E2E、WebNLG 和 DART 上的超参数。

数据集	E2E	WebNLG	DART
训练 (Training)			
Optimizer		AdamW	
Weight Decay	0.01	0.01	0.0
Dropout Prob	0.1	0.1	0.0
Batch Size		8	
# Epoch		5	
Warmup Steps		500	
Learning Rate Schedule		Linear	
Label Smooth	0.1	0.1	0.0
Learning Rate		0.0002	
Adaptation		$r_q = r_v = 4$	
LoRA α		32	
推理 (Inference)			
Beam Size		10	
Length Penalty	0.9	0.8	0.8
no repeat ngram size		4	

表 12: 不同 GPT-3 自适应方法所用的训练超参数。在调整学习率之后，我们对所有数据集使用相同的超参数。

超参数	Fine-Tune	PreEmbed	PreLayer	BitFit	Adapter ^H	LoRA
Optimizer			AdamW			
Batch Size			128			
# Epoch			2			
Warmup Tokens			250,000			
LR Schedule			Linear			
Learning Rate	5.00E-06	5.00E-04	1.00E-04	1.6E-03	1.00E-04	2.00E-04

E 将 LoRA 与前缀微调相结合 (Combining LoRA with Prefix Tuning)

LoRA 可以自然地与现有的基于前缀的方法相结合。在本节中，我们在 WikiSQL 和 MNLI 上评估 LoRA 与前缀微调两种变体的两种组合。

LoRA+PrefixEmbed (LoRA+PE) 将 LoRA 与前缀嵌入微调相结合，其中我们插入 $l_p + l_i$ 个特殊 token，其嵌入被视为可训练参数。关于前缀嵌入微调的更多内容见 5.1 节。

LoRA+PrefixLayer (LoRA+PL) 将 LoRA 与前缀层微调相结合。我们同样插入 $l_p + l_i$ 个特殊 token；然而，我们并不让这些 token 的隐藏表示 (hidden representations) 自然演化，而是在每个 Transformer 块之后用一个与输入无关 (input agnostic) 的向量替换它们。因此，嵌入以及随后的 Transformer 块激活都被视为可训练参数。关于前缀层微调的更多内容见 5.1 节。

在表 15 中，我们展示了 LoRA+PE 和 LoRA+PL 在 WikiSQL 和 MultiNLI 上的评估结果。首先，在 WikiSQL 上，LoRA+PE 显著优于 LoRA 和前缀嵌入微调，这表明 LoRA 与前缀嵌入微调在某种程度上是正交的。在 MultiNLI 上，LoRA+PE 的组合并不比 LoRA 表现更好，这可能是因为 LoRA 自身已经取得了与人类基线相当的性能。其次，我们注意到即便有更多可训练参数，LoRA+PL 的表现也略逊于 LoRA。我们将其归因于：前缀层微调对学习率的选择非常敏感，因而在 LoRA+PL 中使 LoRA 权重的优化更为困难。

F 额外的实证实验 (Additional Empirical Experiments)

F.1 GPT-2 上的额外实验 (Additional Experiments on GPT-2)

我们还遵循 Li & Liang (2021) 的设置，在 DART (Nan et al., 2020) 和 WebNLG (Gardent et al., 2017) 上重复了我们的实验。结果见表 13。与我们在第 5 节中报告的 E2E NLG Challenge 上的结果类似，在可训练参数数量相同的情况下，LoRA 的表现优于或至少与基于前缀的方法持平。

表 13: 在 DART 上采用不同自适应方法的 GPT-2。对所有自适应方法, MET 和 TER 的方差均小于 0.01。

方法	参数量	BLEU $\uparrow$	MET $\uparrow$	TER $\downarrow$
GPT-2 Medium				
Fine-Tune	354M	46.2	0.39	0.46
Adapter ^L	0.37M	42.4	0.36	0.48
Adapter ^L	11M	45.2	0.38	0.46
FT ^{Top2}	24M	41.0	0.34	0.56
PrefLayer	0.35M	46.4	0.38	0.46
LoRA	0.35M	47.1 $\pm$.2	0.39	0.46
GPT-2 Large				
Fine-Tune	774M	47.0	0.39	0.46
Adapter ^L	0.88M	45.7 $\pm$.1	0.38	0.46
Adapter ^L	23M	47.1 $\pm$.1	0.39	0.45
PrefLayer	0.77M	46.7	0.38	0.45
LoRA	0.77M	47.5 $\pm$.1	0.39	0.45

表 14: 在 WebNLG 上采用不同自适应方法的 GPT-2。对我们运行的所有实验, MET 和 TER 的方差均小于 0.01。“U” 表示不可见类别, “S” 表示可见类别, “A” 表示 WebNLG 测试集中的所有类别。

方法	BLEU $\uparrow$			MET $\uparrow$			TER $\downarrow$		
	U	S	A	U	S	A	U	S	A
GPT-2 Medium									
Fine-Tune (354M)	27.7	64.2	46.5	.30	.45	.38	.76	.33	.53
Adapter ^L (0.37M)	45.1	54.5	50.2	.36	.39	.38	.46	.40	.43
Adapter ^L (11M)	48.3	60.4	54.9	.38	.43	.41	.45	.35	.39
FT ^{Top2} (24M)	18.9	53.6	36.0	.23	.38	.31	.99	.49	.72
Prefix (0.35M)	45.6	62.9	55.1	.38	.44	.41	.49	.35	.40
LoRA (0.35M)	46.7 $\pm$.4	62.1 $\pm$.2	55.3 $\pm$.2	.38	.44	.41	.46	.33	.39
GPT-2 Large									
Fine-Tune (774M)	43.1	65.3	55.5	.38	.46	.42	.53	.33	.42
Adapter ^L (0.88M)	49.8 $\pm$.0	61.1 $\pm$.0	56.0 $\pm$.0	.38	.43	.41	.44	.35	.39
Adapter ^L (23M)	49.2 $\pm$.1	64.7 $\pm$.2	57.7 $\pm$.1	.39	.46	.43	.46	.33	.39
Prefix (0.77M)	47.7	63.4	56.3	.39	.45	.42	.48	.34	.40
LoRA (0.77M)	48.4 $\pm$.3	64.0 $\pm$.3	57.0 $\pm$.1	.39	.45	.42	.45	.32	.38

F.2 GPT-3 上的额外实验 (Additional Experiments on GPT-3)

我们在表 15 中给出了 GPT-3 上采用不同自适应方法的额外运行。重点在于厘清性能与可训练参数数量之间的取舍。

F.3 低数据量场景 (Low-Data Regime)

为评估不同自适应方法在低数据量场景下的性能，我们从 MNLI 的完整训练集中随机采样 100、1k 和 10k 个训练样例，构成低数据量的 MNLI- n 任务。在表 16 中，我们展示了不同自适应方法在 MNLI- n 上的性能。令我们惊讶的是，PrefixEmbed 和 PrefixLayer 在 MNLI-100 数据集上表现非常差，其中 PrefixEmbed 仅略好于随机猜测 (37.6% 对 33.3%)。PrefixLayer 的表现优于 PrefixEmbed，但在 MNLI-100 上仍显著差于 Fine-Tune 或 LoRA。随着我们增加训练样例的数量，基于前缀的方法与 LoRA/微调之间的差距变小，这或许表明基于前缀的方法不适用于 GPT-3 中的低数据量任务。在 MNLI-100 和 MNLI-Full 上，LoRA 都取得了比微调更好的性能；考虑到随机种子带来的 (± 0.3) 方差，在 MNLI-1k 和 MNLI-10K 上则取得了相当的结果。

不同自适应方法在 MNLI- n 上的训练超参数报告于表 17。我们在 MNLI-100 集上对 PrefixLayer 使用较小的学习率，因为更大的学习率不会使训练损失下降。

表 15: 在 WikiSQL 和 MNLI 上不同自适应方法的超参数分析。前缀嵌入微调 (PrefixEmbed) 和前缀层微调 (PrefixLayer) 都随着我们增加可训练参数数量而表现更差, 而 LoRA 的性能则趋于稳定。性能以验证准确率度量。

方法	超参数	参数量	WikiSQL	MNLI-m
Fine-Tune	-	175B	73.8	89.5
PrefixEmbed	$l_p = 32, l_i = 8$	0.4 M	55.9	84.9
	$l_p = 64, l_i = 8$	0.9 M	58.7	88.1
	$l_p = 128, l_i = 8$	1.7 M	60.6	88.0
	$l_p = 256, l_i = 8$	3.2 M	63.1	88.6
	$l_p = 512, l_i = 8$	6.4 M	55.9	85.8
PrefixLayer	$l_p = 2, l_i = 2$	5.1 M	68.5	89.2
	$l_p = 8, l_i = 0$	10.1 M	69.8	88.2
	$l_p = 8, l_i = 8$	20.2 M	70.1	89.5
	$l_p = 32, l_i = 4$	44.1 M	66.4	89.6
	$l_p = 64, l_i = 0$	76.1 M	64.9	87.9
Adapter ^H	$r = 1$	7.1 M	71.9	89.8
	$r = 4$	21.2 M	73.2	91.0
	$r = 8$	40.1 M	73.2	91.5
	$r = 16$	77.9 M	73.2	91.5
	$r = 64$	304.4 M	72.6	91.5
LoRA	$r_v = 2$	4.7 M	73.4	91.7
	$r_q = r_v = 1$	4.7 M	73.4	91.3
	$r_q = r_v = 2$	9.4 M	73.3	91.4
	$r_q = r_k = r_v = r_o = 1$	9.4 M	74.1	91.2
	$r_q = r_v = 4$	18.8 M	73.7	91.3
	$r_q = r_k = r_v = r_o = 2$	18.8 M	73.7	91.7
	$r_q = r_v = 8$	37.7 M	73.8	91.6
	$r_q = r_k = r_v = r_o = 4$	37.7 M	74.0	91.7
	$r_q = r_v = 64$	301.9 M	73.6	91.4
	$r_q = r_k = r_v = r_o = 64$	603.8 M	73.9	91.4
LoRA+PE	$r_q = r_v = 8, l_p = 8, l_i = 4$	37.8 M	75.0	91.4
	$r_q = r_v = 32, l_p = 8, l_i = 4$	151.1 M	75.9	91.1
	$r_q = r_v = 64, l_p = 8, l_i = 4$	302.1 M	76.2	91.3
LoRA+PL	$r_q = r_v = 8, l_p = 8, l_i = 4$	52.8 M	72.9	90.2

表 16: 使用 GPT-3 175B 在 MNLI 子集上不同方法的验证准确率。MNLI- n 描述了一个含 n 个训练样例的子集。我们用完整验证集进行评估。与包括微调在内的其他方法相比，LoRA 展现出有利的样本效率 (sample-efficiency)。

方法	MNLI(m)-100	MNLI(m)-1k	MNLI(m)-10k	MNLI(m)-392K
GPT-3 (Fine-Tune)	60.2	85.8	88.9	89.5
GPT-3 (PrefixEmbed)	37.6	75.2	79.5	88.6
GPT-3 (PrefixLayer)	48.3	82.5	85.9	89.6
GPT-3 (LoRA)	63.8	85.6	89.2	91.7

表 17: 不同 GPT-3 自适应方法在 MNLI(m)- n 上所用的超参数。

超参数	自适应 (Adaptation)	MNLI-100	MNLI-1k	MNLI-10K	MNLI-392K
Optimizer	-			AdamW	
Warmup Tokens	-			250,000	
LR Schedule	-			Linear	
Batch Size	-	20	20	100	128
# Epoch	-	40	40	4	2
Learning Rate	FineTune			5.00E-6	
	PrefixEmbed	2.00E-04	2.00E-04	4.00E-04	5.00E-04
	PrefixLayer	5.00E-05	5.00E-05	5.00E-05	1.00E-04
	LoRA			2.00E-4	
Adaptation-Specific	PrefixEmbed l_p	16	32	64	256
	PrefixEmbed l_i			8	
	PrefixTune		$l_p = l_i = 8$		
	LoRA		$r_q = r_v = 8$		

G 度量子空间之间的相似度 (Measuring Similarity Between Subspaces)

在本文中，我们使用度量 $\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\|U_A^{i\top} U_B^j\|_F^2}{\min\{i, j\}}$ 来度量两个列正交 (column orthonormal) 矩阵 $U_A^i \in \mathbb{R}^{d \times i}$ 和 $U_B^j \in \mathbb{R}^{d \times j}$ 之间的子空间相似度，这两个矩阵是通过取 A 和 B 的左奇异矩阵的列得到的。我们指出，这一相似度只不过是度量子空间之间距离的标准投影度量 (Projection Metric) (Ham & Lee, 2008) 的一个反向形式。

具体而言，令 $U_A^{i\top} U_B^j$ 的奇异值为 $\sigma_1, \sigma_2, \dots, \sigma_p$ ，其中 $p = \min\{i, j\}$ 。我们知道投影度量 (Ham

& Lee, 2008) 定义为:

$$d(U_A^i, U_B^j) = \sqrt{p - \sum_{i=1}^p \sigma_i^2} \in [0, \sqrt{p}]$$

而我们的相似度定义为:

$$\phi(A, B, i, j) = \psi(U_A^i, U_B^j) = \frac{\sum_{i=1}^p \sigma_i^2}{p} = \frac{1}{p} (1 - d(U_A^i, U_B^j)^2)$$

这一相似度满足: 如果 U_A^i 和 U_B^j 共享相同的列张成空间 (column span), 那么 $\phi(A, B, i, j) = 1$ 。如果它们完全正交, 那么 $\phi(A, B, i, j) = 0$ 。否则, $\phi(A, B, i, j) \in (0, 1)$ 。

H 关于低秩矩阵的额外实验 (Additional Experiments on Low-Rank Matrices)

我们给出对低秩更新矩阵研究的额外结果。

H.1 LoRA 模块之间的相关性 (Correlation between LoRA Modules)

关于图 3 和图 4 中所呈现的结果如何推广到其他层, 见图 6 和图 7。

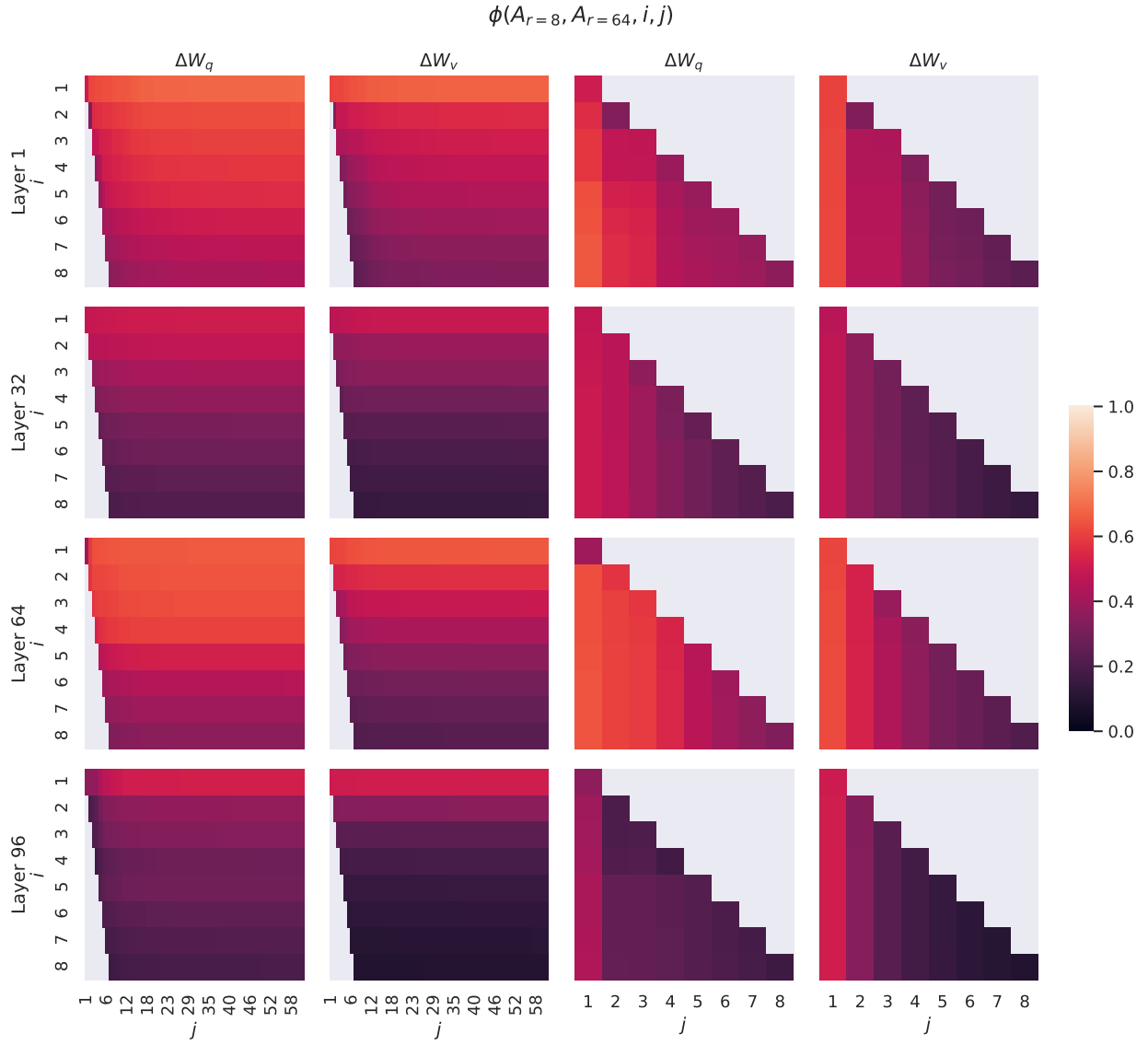


图 6: 对于 96 层 Transformer 中第 1、32、64 和 96 层的 ΔW_q 和 ΔW_v , $A_{r=8}$ 与 $A_{r=64}$ 列向量之间的归一化子空间相似度。

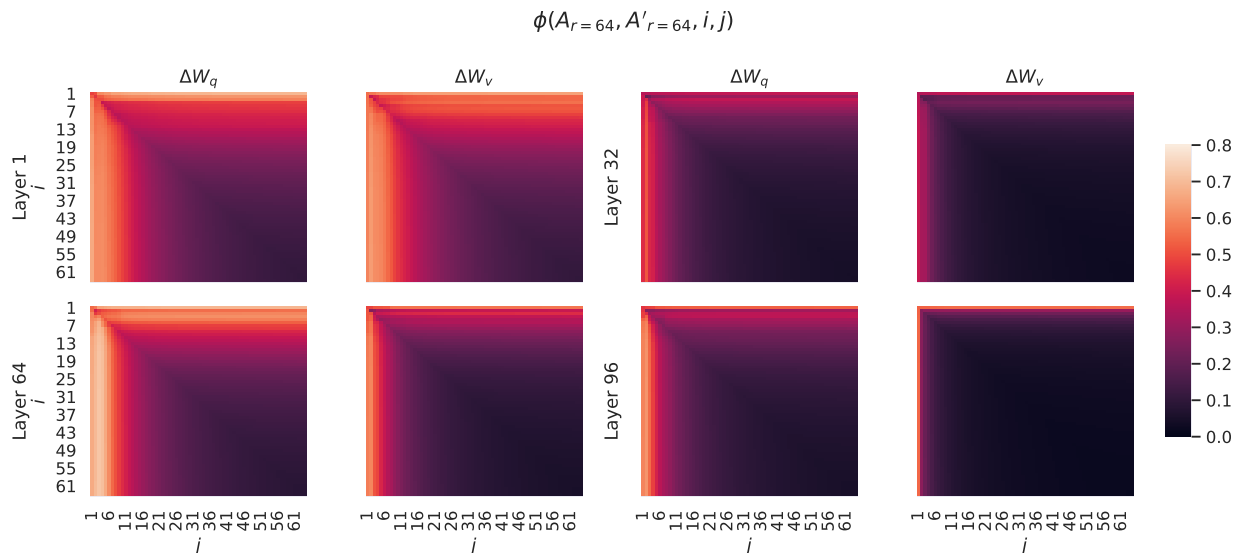


图 7: 对于 96 层 Transformer 中第 1、32、64 和 96 层的 ΔW_q 和 ΔW_v , 来自两个随机种子运行的 $A_{r=64}$ 列向量之间的归一化子空间相似度。

H.2 r 对 GPT-2 的影响 (Effect of r on GPT-2)

我们在 GPT-2 上重复了关于 r 影响的实验 (7.2 节)。以 E2E NLG Challenge 数据集为例, 我们报告训练 26,000 步后, 不同 r 选择所达到的验证损失 (validation loss) 和测试指标。我们在表 18 中给出结果。GPT-2 Medium 的最优秩在 4 到 16 之间, 具体取决于所用指标, 这与 GPT-3 175B 的情况类似。注意, 模型规模与自适应最优秩之间的关系仍是一个开放问题。

H.3 W 与 ΔW 之间的相关性 (Correlation between W and ΔW)

关于 W 与 ΔW 在不同 r 下的归一化子空间相似度, 见图 8。

再次注意, ΔW 并不包含 W 的顶部奇异方向, 因为 ΔW 中前 4 个方向与 W 中前 10% 方向之间的相似度勉强超过 0.2。这提供了证据, 表明 ΔW 包含那些“任务专属 (task-specific)”的、在 W 中本未被强调的方向。

接下来一个有趣的问题是: 为了使模型自适应良好地工作, 我们需要把这些任务专属方向放大得多“强”?

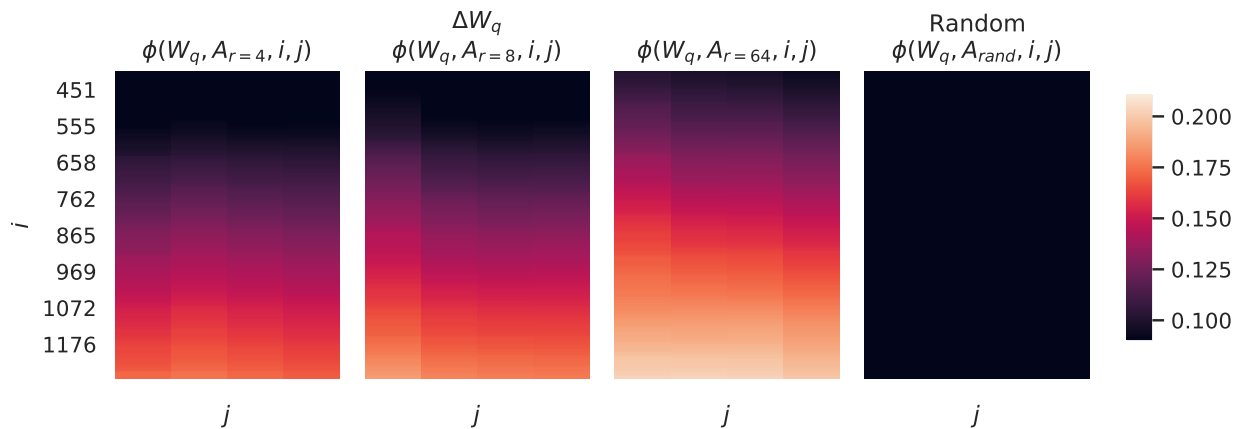


图 8: 在不同 r 下以及一个随机基线下, W_q 的奇异方向与 ΔW_q 的奇异方向之间的归一化子空间相似度。 ΔW_q 放大了那些重要但在 W 中未被强调的方向。具有更大 r 的 ΔW 往往会拾取更多在 W 中已被强调的方向。

H.4 放大因子 (Amplification Factor)

人们可以很自然地将特征放大因子 (feature amplification factor) 考虑为比值 $\frac{\|\Delta W\|_F}{\|U^\top W V^\top\|_F}$, 其中 U 和 V 是 ΔW 的 SVD 分解的左、右奇异矩阵。(回想 $UU^\top W V^\top V$ 给出了 W 到 ΔW 所张成子空间上的“投影”。)

直观上, 当 ΔW 主要包含任务专属方向时, 这一量度量了其中有多少被 ΔW 所放大。如 7.3 节所示, 对于 $r = 4$, 这一放大因子高达 20。换言之, (一般而言) 每层中有四个特征方向 (在来自预训练模型 W 的整个特征空间之中), 需要被一个非常大的因子 20 放大, 才能为特定下游任务达到我们所报告的准确率。并且, 对于每个不同的下游任务, 人们应当预期会有一组非常不同的特征方向被放大。

然而人们可能会注意到, 对于 $r = 64$, 这一放大因子仅约为 2, 这意味着在 $r = 64$ 的 ΔW 中学习到的大多数方向并未被放大很多。这并不令人惊讶, 事实上这 (再次) 提供了证据, 表明表示“任务专属方向” (从而用于模型自适应) 所需的内在秩是低的。相比之下, ΔW 的秩为 4 的版本 (对应于 $r = 4$) 中的那些方向, 被一个大得多的因子 20 所放大。

表 18: 使用 GPT-2 Medium、采用不同秩 r 的 LoRA 在 E2E NLG Challenge 上所达到的验证损失和测试集指标。与 GPT-3 上 $r = 1$ 对许多任务已足够不同的是, 这里性能在验证损失上于 $r = 16$ 达到峰值, 在 BLEU 上于 $r = 4$ 达到峰值, 这表明 GPT-2 Medium 在自适应方面具有与 GPT-3 175B 相似的内在秩。注意我们的某些超参数是在 $r = 4$ 上调整的 (这与另一个基线的参数量相匹配), 因此对于其他 r 的选择可能并非最优。

秩 r	val loss	BLEU	NIST	METEOR	ROUGE L	CIDEr
1	1.23	68.72	8.7215	0.4565	0.7052	2.4329
2	1.21	69.17	8.7413	0.4590	0.7052	2.4639
4	1.18	70.38	8.8439	0.4689	0.7186	2.5349
8	1.17	69.57	8.7457	0.4636	0.7196	2.5196
16	1.16	69.61	8.7483	0.4629	0.7177	2.4985
32	1.16	69.33	8.7736	0.4642	0.7105	2.5255
64	1.16	69.24	8.7174	0.4651	0.7180	2.5070
128	1.16	68.73	8.6718	0.4628	0.7127	2.5030
256	1.16	68.92	8.6982	0.4629	0.7128	2.5012
512	1.16	68.78	8.6857	0.4637	0.7128	2.5025
1024	1.17	69.37	8.7495	0.4659	0.7149	2.5090